



Web Programming Lecture 9

Flexbox

Mahhek Tahir

Understanding Flexbox Layout



What is Flexbox?

- Aligns items in **row or column**

```
.container {  
    display: flex;  
}
```



```
<div class="container">
```

```
  <div class="card">
```

```
    <h3 class="card-title">Card One</h3>
```

```
    <p class="card-text">This is some sample content.</p>
```

```
    <button class="btn">Click Me</button>
```

```
  </div>
```

```
  <div class="card">
```

```
    <h3 class="card-title">Card Two</h3>
```

```
    <p class="card-text">This is some sample content.</p>
```

```
    <button class="btn">Click Me</button>
```

```
  </div>
```

```
  <div class="card">
```

```
    <h3 class="card-title">Card Three</h3>
```

```
    <p class="card-text">This is some sample content.</p>
```

```
    <button class="btn">Click Me</button>
```

```
  </div>
```

```
</div>
```



Applying Flexbox to Parent

- Flexbox is applied to the parent
- All direct children become flex items
- `.card` now behaves as flexible boxes
- Layout becomes horizontal by default

Flexbox always works on the parent.

```
.container {  
    display: flex;  
}
```



Allowing Items to Move

- Default is `nowrap`
- `wrap` allows items to move to next line
- Prevents overflow
- Important for responsive layouts

Simple explanation:

Wrap allows items to break into a new row when space ends.

`gap` creates space between flex items

Cleaner than using margins

Works directly with Flexbox

```
.container {  
    display: flex;  
    flex-wrap: wrap;  
    gap: 20px;  
}
```



Responsiveness

Mobile

```
.card {  
    flex: 1 1 100%;  
}
```

- Takes full width
- One card per row
- Mobile-first approach
- `flex: grow` (1 means Can grow if extra space exists) shrink (Can shrink if space is small) basis (size)

Break it simply:

Start with full width.

Tablet

```
@media (min-width: 768px) {  
    .card {  
        flex: 1 1 calc(50% - 20px);  
    }  
}
```

50% width

Two cards per row

`calc()` adjusts for gap

Responsive behavior

Laptop

```
@media (min-width: 1024px) {  
    .card {  
        flex: 1 1 calc(33.33% - 20px);  
    }  
}
```

33.33% width

Three columns

Fully responsive layout



Flexbox - row vs column

Default Behavior

```
.container {  
  display: flex;  
}
```

Default = `flex-direction: row`
Items arranged **horizontally**

[Home] [News] [Contact] [About]

Using `flex-direction: column`

```
.container {  
  display: flex;  
  flex-direction:  
column;  
}
```

Items arranged **vertically**
(top → bottom)

[Home]
[News]
[Contact]
[About]

```
<div class="container">  
  <div class="box">Home</div>  
  <div class="box">News</div>  
  <div class="box">Contact</div>  
  <div class="box">About</div>  
</div>
```

Without flexbox nav menu?

Display block vs Display - inline

```
html
```

```
<ul>
  <li style="display:inline;">Home</li>
  <li style="display:inline;">About</li>
  <li style="display:inline;">Contact</li>
</ul>
```

Result:

```
nginx
```

```
Home About Contact
```




```
/* Basic Card Styling */
.card{
border:1px solid #eeee; /* Light gray border */
border-radius:10px; /* Rounded corners */
padding:20px; /* Internal spacing */
transition:transform 0.3s ease; /* Smooth hover effect */
}

/* Center Multiple Elements */
.card-title,.card-text{
text-align:center; /* Centers text */
}

/* First Card */
.card:first-child{
border-color:black; /* Styles only first card */
}

/* Last Card */
.card:last-child{
border-color:blue; /* Styles only last card */
}

/* Hover Effect */
.card:hover{
transform:scale(1.25); /* Enlarges card on hover */
}

/* Disabled Button */
button:disabled{
cursor:not-allowed; /* Disabled cursor */
opacity:0.6; /* Faded appearance */
}
```

Data in html

```
<section class="item-list">

  <div class="item-box" data-item-id="501">
    <h4 class="item-name">Laptop Stand</h4>
    <span class="item-cost">$45.00</span>
    <button class="select-btn">Select</button>
  </div>

  <div class="item-box" data-item-id="502">
    <h4 class="item-name">USB Hub</h4>
    <span class="item-cost">$25.00</span>
    <button class="select-btn">Select</button>
  </div>

</section>

<p id="output"></p>
```



Access Clicked Element & Its Data

```
function handleSelect() {  
  
    const button = this; // 'this' refers to clicked button  
    const box = button.closest(".item-box");  
  
    const id = box.dataset.itemId;  
    const name = box.querySelector(".item-name").textContent;  
    const cost = box.querySelector(".item-cost").textContent;  
  
    document.getElementById("output").textContent =  
        `Selected ${name} (ID: ${id}) - ${cost}`;  
}  
  
const buttons = document.querySelectorAll(".select-btn");  
  
buttons.forEach(function(btn) {  
    btn.addEventListener("click", handleSelect);  
});
```

box.getAttribute("data-item-id") will also work

Arrow function version

```
const handleSelect = (event) => {  
  
    const button = event.currentTarget; // Get clicked button  
    const box = button.closest(".item-box");  
  
    const id = box.dataset.itemId;  
    const name = box.querySelector(".item-name").textContent;  
    const cost = box.querySelector(".item-cost").textContent;  
  
    document.getElementById("output").textContent =  
        `Selected ${name} (ID: ${id}) - ${cost}`;  
};  
  
const buttons = document.querySelectorAll(".select-btn");  
  
buttons.forEach(btn => {  
    btn.addEventListener("click", handleSelect);  
});
```



setTimeout

What is setTimeout?

setTimeout runs code
after a specific time delay.

```
const buttons = document.querySelectorAll(".btn");

buttons.forEach(button => {

    button.addEventListener("click", function () {

        const originalText = this.textContent;

        // Change state immediately
        this.textContent = "Processing...";
        this.disabled = true;

        // Restore after 1 second
        setTimeout(() => {
            this.textContent = originalText;
            this.disabled = false;
        }, 1000);

    });

});
```



Async always return a promise

async → always returns a Promise

return value → wrapped inside that Promise

```
async function greet() {  
    return "Hello";  
}
```

```
console.log(greet());
```

```
// Output in console:
```

```
// Promise { "Hello" }
```



When one function depends on other

```
function getFirst() {  
    return Promise.resolve("A");  
}  
  
function addSecond(value) {  
    return Promise.resolve(value + "B");  
}
```

and we have to print AB,
use `await` to get its value.

```
async function run() {  
    const result1 = await getFirst();  
    const result2 = await addSecond(result1);  
  
    console.log(result2);  
}  
  
run();
```

```
function run() {  
    getFirst()  
        .then(addSecond)  
        .then(console.log);  
}  
  
run();
```


Adding and Removing Classes JS



```
const element = document.querySelector(".box");  
element.classList.add("active");
```

Adds the class `active`

Does NOT remove existing classes

Remove a Class

```
element.classList.remove("active");
```

Removes only the class `active`

Other classes remain unchanged

Toggle a Class

```
element.classList.toggle("active");
```

Adds the class if not present

Removes the class if already present

Check if Class Exists

```
element.classList.contains("active");
```

Returns `true` or `false`

Revise

JavaScript String Properties & Methods

Property/Method	Description	Example
length	Returns the number of characters in the string.	"Hello".length ➔ 5
charAt(index)	Returns character at the given index.	"Hello".charAt(1) ➔ 'e'
toUpperCase()	Converts string to uppercase.	"hello".toUpperCase() ➔ 'HELLO'
toLowerCase()	Converts string to lowercase.	"HELLO".toLowerCase() ➔ 'hello'
indexOf(substring)	Finds the first index of a substring.	"Hello".indexOf('l') ➔ 2
slice(start, end)	Extracts a section of the string.	"Hello".slice(1, 4) ➔ 'ello'
split(separator)	Splits string into an array of substrings.	"A,B,C".split(',') ➔ ['A', 'B', 'C']
replace(find, new)	Replaces part of the string.	"Hello".replace('l', 'y') ➔ 'Hey'
trim()	Removes whitespace from both sides.	" Hi ".trim() ➔ 'Hi'
reverse()	Reverses the characters in an array.	['H', 'i'].reverse() ➔ ['i', 'H']

(Use with split/join for strings)



Practice!

In a string Find Words That Start and End With Same Letter

Create 5 vertical bars. Click bar 4, Bars 1–4 become blue, Click bar 4 again, Only bar 4 resets.

Write a JavaScript class called `GameBoard` that represents a rectangular game board of size $m \times n$.

The class should:

1. Store number of rows and columns.
2. Store board values in a 2D array.
3. Include a method `combine(otherBoard)` that:
4. If the board sizes do not match, display: